



Seguridad de la plataforma

18 de Julio 2026

Tabla de contenido

1. Introducción.....	4
1.1 Objetivo	4
1.2 Alcance.....	4
2. Activos y modelo de amenazas	4
2.1 Activos que deben protegerse	4
2.2 Modelo general de amenazas	5
3. Autenticación	5
3.1 Estrategia de autenticación	5
3.2 Registro e inicio de sesión	5
3.3 Verificación de correo.....	5
3.4 Recuperación de contraseña.....	5
3.5 Manejo de tokens y sesiones.....	5
3.6 Cierre de sesión	5
4. Autorización.....	6
4.1 Estrategia de autorización.....	6
4.2 Roles y permisos.....	6
4.3 RBAC y principio de mínimo privilegio	6
5. Protección de APIs y microservicios.....	6
5.1 Validación en el API Gateway	6
5.2 Validación en cada microservicio	6
5.3 HTTPS, CORS y rate limiting.....	7
5.4 Validación de entradas y protección contra inyección.....	7
5.5 Headers de seguridad	7
5.6 Protección de endpoints internos y seguridad entre microservicios	7
5.7 Seguridad de RabbitMQ.....	7
6. Gestión de secretos y entornos.....	7
6.1 Gestión de secretos	7

6.2 Variables de entorno.....	8
6.3 Separación DEV y PROD	8
7. Seguridad de datos y pagos	8
7.1 Seguridad de datos	8
7.2 Seguridad de pagos	8
7.3 Seguridad del monedero	8
7.4 Seguridad del QR	8
8. Auditoría	9
9. Matriz de riesgos y relación con OWASP	9
9.1 Matriz de riesgos por microservicio	9
9.2 Controles de mitigación recomendados	9
9.3 Relación con OWASP	10
10. Riesgos residuales y mejoras futuras	10
10.1 Riesgos residuales	10
10.2 Mejoras futuras.....	10

1. Introducción

Este documento describe cómo ECIExpress protege sus activos y usuarios, y cuáles son sus principales riesgos de seguridad. ECIExpress es una plataforma de marketplace universitario construida como un sistema de 8 microservicios en NestJS, con billetera digital recargable, autenticación centralizada, mensajería asíncrona entre servicios y pagos a través de Wompi.

1.1 Objetivo

Documentar las decisiones y controles de seguridad implementados en la plataforma, la estrategia de autenticación y autorización, la protección de las APIs y la comunicación entre microservicios, la gestión de secretos, y el estado real de los riesgos de seguridad identificados a la fecha de este documento.

1.2 Alcance

Cubre los 8 microservicios de la plataforma, Identity, Products, Orders, Financial, Fulfillment, Notifications, Reporting y API Gateway, así como el bus de eventos asíncrono y la comunicación entre servicios. No cubre la infraestructura de red de Azure en producción, que se documenta en el material de arquitectura y despliegue del proyecto.

2. Activos y modelo de amenazas

2.1 Activos que deben protegerse

- Credenciales e identidad de los usuarios, compradores, vendedores y administradores.
- Saldo y movimientos de la billetera digital de cada usuario.
- Datos de pago y comprobantes procesados a través de Wompi.
- Catálogo, inventario y promociones de cada tienda.
- Pedidos, su historial y su estado de entrega.
- Secretos de integración, credenciales de Wompi, claves de Firebase y cadenas de conexión a bases de datos.
- Imágenes y archivos almacenados, fotos de perfil, imágenes de tiendas, productos, comprobantes y códigos QR.

2.2 Modelo general de amenazas

El proyecto no cuenta todavía con un modelo de amenazas formal ni con una matriz de riesgos construida con una metodología específica, como STRIDE. Lo que sí existe es una auditoría de seguridad basada en revisión estática de código contra las categorías de OWASP, cuyos resultados se documentan en la sección 9 de este documento.

3. Autenticación

3.1 Estrategia de autenticación

La autenticación de usuarios se delega en Firebase Auth. El API Gateway valida el token de Firebase en el borde de la plataforma, traduce el UID de Firebase a un UUID interno, e inyecta el encabezado X-User-Id hacia los microservicios internos.

3.2 Registro e inicio de sesión

El registro se hace por correo y contraseña, y también está disponible el inicio de sesión con Google, ambos gestionados por Firebase Auth.

3.3 Verificación de correo

La verificación de correo está implementada a través de Firebase.

3.4 Recuperación de contraseña

La recuperación de contraseña está implementada y funciona a través del flujo de recuperación nativo de Firebase.

3.5 Manejo de tokens y sesiones

El manejo de tokens, la expiración de sesiones y el refresco de tokens con un tiempo de vida mayor son gestionados directamente por Firebase, la plataforma no implementa su propia lógica de expiración o rotación de tokens. El microservicio de Identity valida además que el token no esté expirado ni revocado, y verifica un identificador de sesión, X-Session-Id, sobre cada solicitud autenticada.

3.6 Cierre de sesión

El cierre de sesión se apoya en la invalidación de sesión de Firebase en el cliente. La verificación de token expirado o revocado en el microservicio de Identity refuerza este comportamiento del lado del servidor.

4. Autorización

4.1 Estrategia de autorización

La autorización combina el control en el borde, en el API Gateway, con validaciones adicionales dentro de cada microservicio. El Gateway inyecta la identidad verificada del usuario, y cada microservicio decide qué puede hacer ese usuario según su rol y, cuando aplica, su relación de propiedad con el recurso, por ejemplo su tienda.

4.2 Roles y permisos

La plataforma define 4 roles:

Comprador (buyer): navega el marketplace, realiza pedidos y usa su billetera.

Vendedor (vendor): administra su tienda, catálogo, inventario y pedidos recibidos.

Administrador (administrador): administra la plataforma completa.

Analista: además de lo que puede ver un administrador, tiene acceso a la pestaña de analytics, con datos de login y monitoreo.

4.3 RBAC y principio de mínimo privilegio

El control de acceso por rol, RBAC, se aplica principalmente a través de guards en cada microservicio, apoyados en el encabezado de rol que reenvía el Gateway. La auditoría de seguridad de la sección 9 identificó que, en varios microservicios, esta autorización depende únicamente del encabezado reenviado por el Gateway, sin una verificación criptográfica adicional del lado del microservicio, lo que se documenta como hallazgo abierto.

5. Protección de APIs y microservicios

5.1 Validación en el API Gateway

El Gateway valida el token de Firebase, enruta las peticiones a cada microservicio, e inyecta la identidad verificada en los encabezados internos antes de reenviar la solicitud.

5.2 Validación en cada microservicio

Cada microservicio aplica su propio ValidationPipe de NestJS sobre los DTOs de entrada. La auditoría de la sección 9 identificó puntos donde esa validación es inconsistente o está ausente, y se documentan allí como hallazgos abiertos.

5.3 HTTPS, CORS y rate limiting

El tráfico se sirve sobre HTTPS. El Gateway tiene configurado un throttler general de solicitudes por IP. La configuración de CORS varía entre microservicios, en varios casos usa una política abierta en lugar de una lista blanca de orígenes, lo cual se documenta como hallazgo en la sección 9.

5.4 Validación de entradas y protección contra inyección

La validación de entradas se apoya en class-validator y ValidationPipe en la mayoría de los endpoints. El webhook de Wompi es una excepción deliberada, recibe el cuerpo crudo sin un DTO tipado, porque un ValidationPipe con whitelist en true eliminaría campos necesarios para verificar la firma de la notificación antes de que esta pueda validarse.

5.5 Headers de seguridad

El endurecimiento de encabezados de seguridad HTTP, como los que ofrece una librería tipo helmet, no se documentó como un control explícito del proyecto a la fecha de este documento.

5.6 Protección de endpoints internos y seguridad entre microservicios

El Gateway bloquea el acceso externo directo a las rutas internas y a la documentación Swagger de los microservicios internos. Sin embargo, la comunicación entre el Gateway y los microservicios, y entre microservicios, se apoya en encabezados internos que no están firmados, por ejemplo un valor estático como único mecanismo de identificación entre servicios. La auditoría de la sección 9 documenta esto como un riesgo de autenticación servicio a servicio.

5.7 Seguridad de RabbitMQ

La comunicación asíncrona usa Azure Service Bus. RabbitMQ se usó en una etapa anterior del proyecto y ya se dejó de usar. Los eventos siguen la convención servicio punto entidad punto acción, con un envelope estándar que incluye idempotencyKey, correlationId y eventVersion, lo que da trazabilidad y evita el reprocesamiento indebido de un mismo evento.

6. Gestión de secretos y entornos

6.1 Gestión de secretos

Los secretos del proyecto se manejan por variables de entorno, no están escritos en el código y los archivos .env están excluidos del control de versiones. La auditoría de la sección 9 encontró un caso donde secretos de integración con Wompi quedaban expuestos en los registros de la aplicación, logs, en el microservicio Financiamiento, lo cual se documenta como hallazgo crítico abierto.

6.2 Variables de entorno

Cada microservicio recibe su configuración sensible por variables de entorno, siguiendo el mismo patrón en todos los servicios del proyecto.

6.3 Separación DEV y PROD

El proyecto usa proveedores separados para el entorno de MVP, Render, NeonDB y CloudAMQP, con planes de migración hacia Azure Container Apps, Azure PostgreSQL Flexible Server y Azure Service Bus para producción.

7. Seguridad de datos y pagos

7.1 Seguridad de datos

TypeORM se configura siempre con synchronize en false, todo cambio de esquema de base de datos pasa por migraciones controladas, para evitar cambios accidentales de estructura en producción.

7.2 Seguridad de pagos

Los pagos se procesan a través de Wompi, con soporte para Nequi, Daviplata, PSE, Tarjeta, Bancolombia y Bancolombia QR. El webhook de confirmación de pago valida correctamente la firma SHA256 enviada por Wompi antes de aceptar una notificación de pago.

7.3 Seguridad del monedero

Las transacciones de la billetera digital se manejan de forma idempotente en el microservicio Financiamiento, con actualizaciones condicionales y restricciones de unicidad para evitar el doble procesamiento de un mismo movimiento.

7.4 Seguridad del QR

Los códigos QR de entrega se generan en el microservicio de Fulfillment, con manejo de concurrencia mediante bloqueo transaccional para evitar que un mismo código se valide dos veces.

8. Auditoría

El proyecto cuenta con filtros globales de manejo de excepciones en varios microservicios, que evitan exponer trazas internas del servidor en las respuestas al cliente. El microservicio de Reporting centraliza el logging y el monitoreo de la plataforma a través de su módulo de analytics, que expone indicadores, dashboards y datos de login accesibles para el rol de analista. No se documentó a la fecha un sistema independiente de auditoría o registro específico de accesos administrativos, separado de este monitoreo general.

9. Matriz de riesgos y relación con OWASP

El equipo ejecutó una auditoría de seguridad y rendimiento con Claude Code sobre los 8 microservicios del proyecto, mediante revisión estática de código contra las categorías vigentes de OWASP. La siguiente tabla es una versión condensada por microservicio, sin el detalle técnico de archivo y línea, ese detalle se conserva como documentación interna de soporte y no se incluye en este documento.

9.1 Matriz de riesgos por microservicio

Microservicio	Crít.	Alta	Media	Baja	Categorías OWASP principales
Identity	0	1	4	2	Autenticación rota, autorización a nivel de función
Products	2	3	4	1	BOLA / IDOR, configuración de seguridad
Orders	2	4	2	0	BOLA / IDOR, autorización a nivel de función, autenticación rota
Financial	1	2	3	1	Exposición de datos criptográficos, BOLA / IDOR
Fulfillment	0	0	2	2	Consumo de recursos, autenticación rota
Notifications	1	2	4	1	Inyección, BOLA / IDOR
Reporting	0	0	2	3	Gestión de inventario de APIs, configuración de seguridad
API Gateway	0	4	3	0	Configuración de seguridad, consumo de recursos, autenticación rota

9.2 Controles de mitigación recomendados

- Validar la propiedad del recurso, por ejemplo la tienda o el pedido, antes de permitir una modificación, en lugar de confiar solo en el identificador recibido.
- Eliminar de los registros cualquier valor derivado de un secreto de integración.

- Reemplazar los encabezados internos estáticos entre Gateway y microservicios por un secreto firmado o rotatable, o por autenticación mutua.
- Definir una lista blanca explícita de orígenes para CORS en cada microservicio.
- Aplicar paginación obligatoria en los listados que hoy devuelven todos los registros.
- Sanear las entradas usadas en el envío de correos y plantillas de notificación.

9.3 Relación con OWASP

Los hallazgos de la auditoría se agrupan principalmente en las categorías de API Security Top 10 de OWASP: autorización a nivel de objeto rota, BOLA; autenticación rota; autorización a nivel de función rota; consumo de recursos sin restricción; configuración de seguridad incorrecta; e inyección. No se ha hecho todavía un mapeo formal completo contra el listado vigente de OWASP para todas las categorías restantes.

10. Riesgos residuales y mejoras futuras

10.1 Riesgos residuales

- Los hallazgos críticos y altos de la auditoría de seguridad, sección 9, siguen abiertos.
- El certificado de la plataforma en el entorno de MVP es autofirmado, el navegador no confía en él de forma nativa.
- Los microservicios corren hoy con una sola réplica cada uno, sin redundancia por localización.
- La disponibilidad continua del módulo de IA de la plataforma no tiene todavía una estrategia definida de escalado o límite de uso.

10.2 Mejoras futuras

- Remediar los hallazgos críticos y altos documentados en la auditoría de seguridad.
- Comprar un dominio propio para la plataforma.
- Reemplazar el certificado autofirmado por uno confiado por el navegador.
- Escalar los microservicios en Azure Container Apps a varias réplicas por localización, en lugar de una sola réplica por servicio.

- Ofuscar los nombres públicos de los microservicios, para que un atacante no pueda identificar fácilmente cuál servicio es el más crítico a atacar.
- Realizar pruebas de penetración manuales y escaneo dinámico de vulnerabilidades sobre el entorno desplegado.